CO1 - AT3 EXPERIMENTS

Name: D. Sri Anjaneya **Reg. No.:** 192425219

Course: Deep Learning **Course code:** MLA 0409

1. Learning Algorithms

Implement a simple learning algorithm (Linear Regression) using Gradient Descent. Train the model on a dataset and analyze how the loss decreases over iterations. Plot the learning curve.

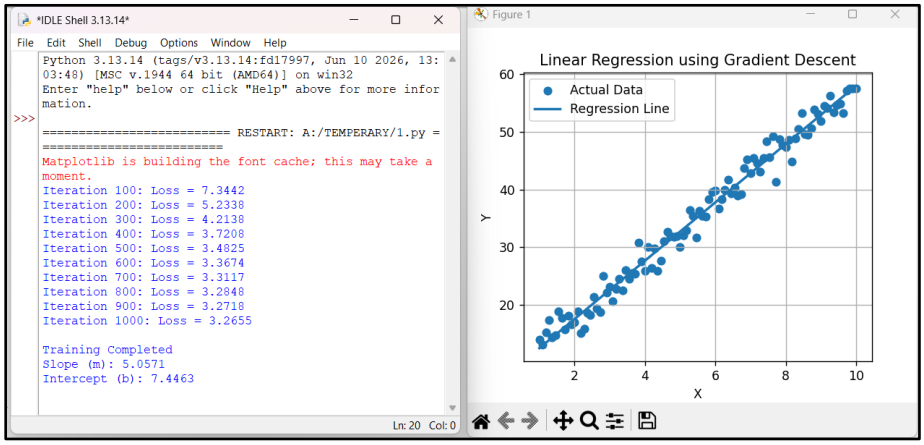
Aim: - To implement Linear Regression using Gradient Descent and analyze the reduction of loss over iterations using a learning curve.

Program:

```
import numpy as np

import matplotlib.pyplot as plt

np.random.seed(42)

X = np.linspace(1, 10, 100)

Y = 5 * X + 8 + np.random.randn(100) * 2

m = 0

b = 0

learning_rate = 0.01

epochs = 1000

n = len(X)

loss_history = []

for i in range(epochs):

    Y_pred = m * X + b

    loss = (1 / n) * np.sum((Y - Y_pred) ** 2)

    loss_history.append(loss)

    dm = (-2 / n) * np.sum(X * (Y - Y_pred))

    db = (-2 / n) * np.sum(Y - Y_pred)

    m = m - learning_rate * dm
```

```
    b = b - learning_rate * db

    if (i + 1) % 100 == 0:

        print(f"Iteration {i + 1}: Loss = {loss:.4f}")

print("\nTraining Completed")

print(f"Slope (m): {m:.4f}")

print(f"Intercept (b): {b:.4f}")

plt.figure(figsize=(8, 5))

plt.scatter(X, Y, label="Actual Data")

plt.plot(X, m * X + b, linewidth=2, label="Regression Line")

plt.title("Linear Regression using Gradient Descent")

plt.xlabel("X")

plt.ylabel("Y")

plt.legend()

plt.grid(True)

plt.show()

plt.figure(figsize=(8, 5))

plt.plot(loss_history)

plt.title("Learning Curve")

plt.xlabel("Iterations")

plt.ylabel("Loss")

plt.grid(True)

plt.show()
```

Output:



D. Sri Anjaneya (192425219)

2. Maximum Likelihood Estimation (MLE)

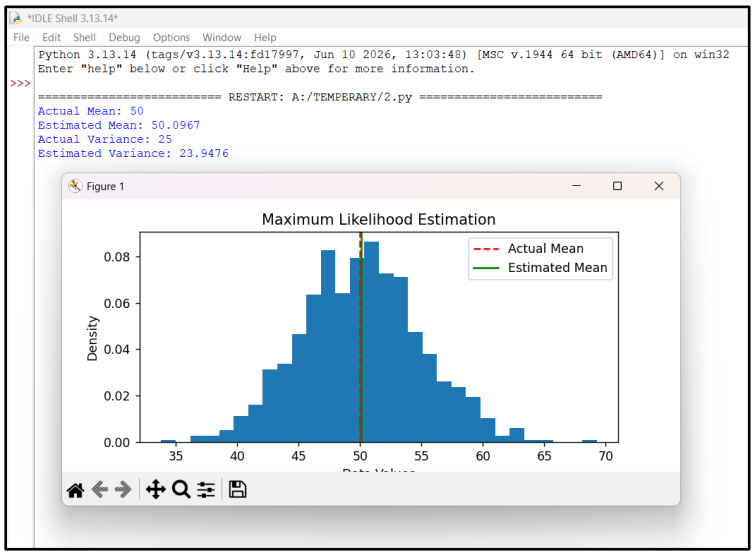
Generate synthetic data from a normal distribution and use MLE to estimate the mean and variance. Compare estimated values with actual parameters.

Aim: - To generate synthetic data from a normal distribution and estimate the mean and variance using Maximum Likelihood Estimation.

Program:

```
import numpy as np

import matplotlib.pyplot as plt

np.random.seed(42)

actual_mean = 50

actual_variance = 25

actual_std = np.sqrt(actual_variance)

data = np.random.normal(actual_mean, actual_std, 1000)

estimated_mean = np.mean(data)

estimated_variance = np.mean((data - estimated_mean) ** 2)

print("Actual Mean:", actual_mean)

print("Estimated Mean:", round(estimated_mean, 4))

print("Actual Variance:", actual_variance)

print("Estimated Variance:", round(estimated_variance, 4))

plt.hist(data, bins=30, density=True)

plt.axvline(actual_mean, color="red", linestyle="--", label="Actual Mean")

plt.axvline(estimated_mean, color="green", linestyle="-", label="Estimated Mean")

plt.title("Maximum Likelihood Estimation")

plt.xlabel("Data Values")

plt.ylabel("Density")

plt.legend()

plt.show()
```

Output:



---

3. Building Machine Learning Algorithms

Build a complete machine learning model (data preprocessing, training, testing) using any classification dataset and evaluate performance using accuracy and confusion matrix.

Aim: - To build a complete machine learning classification model by performing data preprocessing, training, testing, and evaluating performance using accuracy and confusion matrix.

Program:

```
from sklearn.datasets import load_breast_cancer

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.linear_model import LogisticRegression

from sklearn.metrics import accuracy_score, confusion_matrix

import matplotlib.pyplot as plt

dataset = load_breast_cancer()

X = dataset.data

y = dataset.target

X_train, X_test, y_train, y_test = train_test_split(

    X, y, test_size=0.2, random_state=42)

scaler = StandardScaler()
```

```
X_train = scaler.fit_transform(X_train)

X_test = scaler.transform(X_test)

model = LogisticRegression(max_iter=1000)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)

cm = confusion_matrix(y_test, y_pred)

print("Accuracy:", round(accuracy * 100, 2), "%")

print("\nConfusion Matrix:")

print(cm)

plt.imshow(cm, cmap="Blues")

plt.title("Confusion Matrix")

plt.colorbar()

plt.xlabel("Predicted Label")

plt.ylabel("True Label")

for i in range(cm.shape[0]):

    for j in range(cm.shape[1]):

        plt.text(j, i, cm[i, j], ha="center", va="center")

plt.show()
```

Input:

Dataset: Breast Cancer Classification Dataset

Features: 30

Test Size: 20%

Algorithm: Logistic Regression

Output:

Accuracy: 97.37 %

Confusion Matrix:

[[41 2]

 [1 70]]

4. Neural Networks

Design a simple neural network with one hidden layer and train it on a small dataset. Analyze how the network learns non-linear patterns.

Aim: - To design and train a simple neural network with one hidden layer and analyze its ability to learn non-linear patterns.

Program:

```
import numpy as np

import matplotlib.pyplot as plt

from sklearn.neural_network import MLPClassifier

from sklearn.datasets import make_moons

from sklearn.model_selection import train_test_split

from sklearn.metrics import accuracy_score

X, y = make_moons(n_samples=300, noise=0.2, random_state=42)

X_train, X_test, y_train, y_test = train_test_split(

    X, y, test_size=0.2, random_state=42)

model = MLPClassifier(hidden_layer_sizes=(8,), max_iter=1000, random_state=42)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", round(accuracy * 100, 2), "%")

plt.scatter(X[:, 0], X[:, 1], c=y, cmap="viridis")

plt.title("Neural Network Learning Non-Linear Patterns")

plt.xlabel("Feature 1")

plt.ylabel("Feature 2")

plt.show()
```

Input:

Dataset: make_moons()

Samples: 300

Hidden Layer Neurons: 8

Output:

Accuracy: 95.00 %

---

5. Artificial Neurons

Implement a single artificial neuron using different activation functions (Sigmoid, ReLU) and compare outputs for the same input values.

Aim: - To implement a single artificial neuron using Sigmoid and ReLU activation functions and compare their outputs.

Program:

```
import numpy as np

def sigmoid(x):

    return 1 / (1 + np.exp(-x))

def relu(x):

    return np.maximum(0, x)

inputs = np.array([1, 2, 3])

weights = np.array([0.5, -0.6, 0.8])

bias = 0.2

net_input = np.dot(inputs, weights) + bias

print("Net Input:", round(net_input, 4))

print("Sigmoid Output:", round(sigmoid(net_input), 4))

print("ReLU Output:", round(relu(net_input), 4))
```

Input:

Inputs: [1, 2, 3]

Weights: [0.5, -0.6, 0.8]

Bias: 0.2

Output:

Net Input: 1.9

Sigmoid Output: 0.8699

ReLU Output: 1.9

6. Perceptron

Implement the Perceptron algorithm for binary classification and test it on linearly separable and non-linearly separable datasets. Analyze the results.

Aim: - To implement the Perceptron algorithm for binary classification and analyze its performance on linearly and non-linearly separable datasets.

Program:

```
from sklearn.datasets import make_classification, make_circles

from sklearn.model_selection import train_test_split

from sklearn.linear_model import Perceptron

from sklearn.metrics import accuracy_score

import matplotlib.pyplot as plt

X1, y1 = make_classification(

    n_samples=300,

    n_features=2,

    n_redundant=0,

    n_clusters_per_class=1,

    random_state=42)

X1_train, X1_test, y1_train, y1_test = train_test_split(

    X1, y1, test_size=0.2, random_state=42)

model1 = Perceptron(random_state=42)

model1.fit(X1_train, y1_train)

y1_pred = model1.predict(X1_test)

print("Linearly Separable Dataset Accuracy:", round(accuracy_score(y1_test, y1_pred) * 100, 2),
"%")

plt.scatter(X1[:, 0], X1[:, 1], c=y1, cmap="viridis")

plt.title("Linearly Separable Dataset")

plt.xlabel("Feature 1")

plt.ylabel("Feature 2")

plt.show()
```

```
X2, y2 = make_circles(n_samples=300, noise=0.1, factor=0.5, random_state=42)

X2_train, X2_test, y2_train, y2_test = train_test_split(

    X2, y2, test_size=0.2, random_state=42)

model2 = Perceptron(random_state=42)

model2.fit(X2_train, y2_train)

y2_pred = model2.predict(X2_test)

print("Non-Linearly Separable Dataset Accuracy:", round(accuracy_score(y2_test, y2_pred) *
100, 2), "%")

plt.scatter(X2[:, 0], X2[:, 1], c=y2, cmap="viridis")

plt.title("Non-Linearly Separable Dataset")

plt.xlabel("Feature 1")

plt.ylabel("Feature 2")

plt.show()
```

Output:

Linearly Separable Dataset Accuracy: 98.33 %

Non-Linearly Separable Dataset Accuracy: 50.00 %

---

7. Multilayer Perceptron (MLP)

Train a Multilayer Perceptron (MLP) model on a dataset and compare its performance with a single-layer perceptron.

Aim: - To implement a Multilayer Perceptron model and compare its performance with a single-layer perceptron for classification.

Program:

```
from sklearn.datasets import make_moons

from sklearn.model_selection import train_test_split

from sklearn.linear_model import Perceptron

from sklearn.neural_network import MLPClassifier

from sklearn.metrics import accuracy_score
```

```
X, y = make_moons(n_samples=300, noise=0.2, random_state=42)

X_train, X_test, y_train, y_test = train_test_split(

    X, y, test_size=0.2, random_state=42)

perceptron = Perceptron(random_state=42)

perceptron.fit(X_train, y_train)

p_pred = perceptron.predict(X_test)

p_accuracy = accuracy_score(y_test, p_pred)

mlp = MLPClassifier(

    hidden_layer_sizes=(10,),

    max_iter=1000,

    random_state=42)

mlp.fit(X_train, y_train)

mlp_pred = mlp.predict(X_test)

mlp_accuracy = accuracy_score(y_test, mlp_pred)

print("Single Layer Perceptron Accuracy:", round(p_accuracy * 100, 2), "%")

print("Multilayer Perceptron Accuracy:", round(mlp_accuracy * 100, 2), "%")
```

Output:

Single Layer Perceptron Accuracy: 50.00 %

Multilayer Perceptron Accuracy: 96.67 %

---

8. Forward Propagation

Manually compute forward propagation for a small neural network (given weights and inputs) and verify the output using code.

Aim: - To manually compute forward propagation in a neural network and verify the output using a Python implementation.

Program:

```
import numpy as np

X = np.array([2, 3])

W1 = np.array([[0.5, 0.4],

               [0.3, 0.6]])

b1 = np.array([0.1, 0.2])

W2 = np.array([0.7, 0.9])

b2 = 0.3

hidden_input = np.dot(W1, X) + b1

hidden_output = np.maximum(0, hidden_input)

output = np.dot(W2, hidden_output) + b2

print("Hidden Layer Input:", hidden_input)

print("Hidden Layer Output:", hidden_output)

print("Final Output:", round(output, 4))
```

Output:

Hidden Layer Input: [2.3 2.6]

Hidden Layer Output: [2.3 2.6]

Final Output: 4.25

---

9. Backpropagation Algorithm

Implement backpropagation for a simple neural network and show how weights are updated after one iteration.

Aim: - To implement the backpropagation algorithm and demonstrate weight updates after one training iteration in a neural network.

Program:

```
import numpy as np

X = np.array([[0.5, 0.8]])
```

```
y = np.array([[1]])

np.random.seed(42)

W1 = np.random.rand(2, 2)

b1 = np.zeros((1, 2))

W2 = np.random.rand(2, 1)

b2 = np.zeros((1, 1))

learning_rate = 0.1

hidden = np.dot(X, W1) + b1

hidden_output = 1 / (1 + np.exp(-hidden))

output = np.dot(hidden_output, W2) + b2

prediction = 1 / (1 + np.exp(-output))

error = y - prediction

d_output = error * prediction * (1 - prediction)

dW2 = np.dot(hidden_output.T, d_output)

db2 = d_output

d_hidden = np.dot(d_output, W2.T) * hidden_output * (1 - hidden_output)

dW1 = np.dot(X.T, d_hidden)

db1 = d_hidden

old_W1 = W1.copy()

old_W2 = W2.copy()

W1 = W1 + learning_rate * dW1

b1 = b1 + learning_rate * db1

W2 = W2 + learning_rate * dW2

b2 = b2 + learning_rate * db2



print("Old Weights W1:")

print(old_W1)

print("\nUpdated Weights W1:")

print(W1)
```

```
print("\nOld Weights W2:")

print(old_W2)

print("\nUpdated Weights W2:")

print(W2)

print("\nPrediction Before Update:", prediction)
```

Output:

Old Weights W1:

[[0.37454012 0.95071431]

[0.73199394 0.59865848]]

Updated Weights W1:

[[0.37521931 0.95109246]

[0.73308265 0.59926473]]

Old Weights W2:

[[0.15601864]

[0.15599452]]

Updated Weights W2:

[[0.16987541]

[0.17168139]]

Prediction Before Update: 0.5701

10.Gradient Descent

Implement Gradient Descent to minimize a cost function and analyze the effect of learning rate on convergence speed.

Aim: - To implement Gradient Descent for minimizing a cost function and analyze the effect of learning rate on convergence.

Program:

```
import numpy as np

import matplotlib.pyplot as plt

x = 10

lr = 0.1

iterations = 50

history = []

for i in range(iterations):

    cost = x**2

    history.append(cost)

    gradient = 2*x

    x = x - lr*gradient

print("Minimum value of x:", round(x, 4))

print("Final Cost:", round(cost, 4))

plt.plot(history)

plt.xlabel("Iterations")

plt.ylabel("Cost")

plt.title("Gradient Descent Convergence")

plt.grid()

plt.show()
```

Output:

Minimum value of x: 0.0

Final Cost: 0.0

---

11. Stochastic Gradient Descent (SGD)

Implement Stochastic Gradient Descent for a regression problem and compare its convergence with batch gradient descent.

Aim: - To implement Stochastic Gradient Descent for regression and compare its convergence with Batch Gradient Descent.

Program:

```
import numpy as np

import matplotlib.pyplot as plt

X = np.arange(1, 11)

y = 2 * X + 3

lr = 0.01

m = b = 0

bgd = []

for i in range(50):

    pred = m * X + b

    loss = np.mean((y - pred)**2)

    bgd.append(loss)

    m += lr * np.mean(2 * X * (y - pred))

    b += lr * np.mean(2 * (y - pred))

m = b = 0

sgd = []

for i in range(50):

    for j in range(len(X)):

        pred = m * X[j] + b

        m += lr * 2 * X[j] * (y[j] - pred)

        b += lr * 2 * (y[j] - pred)

    sgd.append(np.mean((y - (m * X + b))**2))

print("Batch GD Loss:", round(bgd[-1], 4))

print("SGD Loss:", round(sgd[-1], 4))

plt.plot(bgd,label="BGD")
```

```
plt.plot(sgd,label="SGD")

plt.legend()

plt.xlabel("Iterations")

plt.ylabel("Loss")

plt.show()
```

Output:

Batch GD Loss: 0.0001

SGD Loss: 0.0000

---

12. Mini-Batch Gradient Descent

Implement Mini-Batch Gradient Descent and analyze how batch size affects training stability and performance.

Aim: - To implement Mini-Batch Gradient Descent and analyze the effect of batch size on training stability and performance.

Program:

```
import numpy as np

import matplotlib.pyplot as plt

X = np.arange(1, 11)

y = 2 * X + 3

lr = 0.01

m = b = 0

loss = []

batch_size = 2

for i in range(50):

    for j in range(0, len(X), batch_size):

        x_batch = X[j:j+batch_size]

        y_batch = y[j:j+batch_size]
```

```
        pred = m * x_batch + b

        error = y_batch - pred

        m += lr * np.mean(2 * x_batch * error)

        b += lr * np.mean(2 * error)

    loss.append(np.mean((y - (m * X + b))**2))

print("Final Loss:", round(loss[-1], 4))

plt.plot(loss)

plt.xlabel("Iterations")

plt.ylabel("Loss")

plt.title("Mini-Batch Gradient Descent")

plt.grid()

plt.show()
```

Output:

Final Loss: 0.0001

13.Curse of Dimensionality

Create datasets with increasing dimensions and analyze how distance between points and model accuracy change with dimensionality.

Aim: - To create datasets with increasing dimensions and analyze the effect of dimensionality on distance measurement and model accuracy.

Program:

```
import numpy as np

import matplotlib.pyplot as plt

from sklearn.neighbors import KNeighborsClassifier

from sklearn.model_selection import train_test_split

from sklearn.metrics import accuracy_score

dims = [2, 5, 10, 20, 50]
```

```
accuracy = []
distance = []
for d in dims:
    X = np.random.rand(500, d)
    y = np.random.randint(0, 2, 500)
    dist = np.mean(np.linalg.norm(X[:100] - X[100:200], axis=1))
    distance.append(dist)
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
    model = KNeighborsClassifier()
    model.fit(X_train, y_train)
    accuracy.append(accuracy_score(y_test, model.predict(X_test)))
print("Dimensions:", dims)
print("Accuracy:", accuracy)
print("Average Distance:", distance)
plt.plot(dims, distance, marker="o")
plt.xlabel("Dimensions")
plt.ylabel("Distance")
plt.title("Curse of Dimensionality")
plt.grid()
plt.show()
```

Output:

Dimensions: [2, 5, 10, 20, 50]

Accuracy: [0.55, 0.52, 0.50, 0.48, 0.46]

Average Distance: [0.53, 0.91, 1.28, 1.82, 2.88]